30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 9

Testing Strategies — Part 1: Unit Tests, Test Doubles, Coroutines & TDD



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 9

Why testing matters in 2026

Testing questions reveal whether you can **change code without fear**. Interviewers no longer ask "do you write tests?" — they ask *what* you test, *where*, and with *which* double:

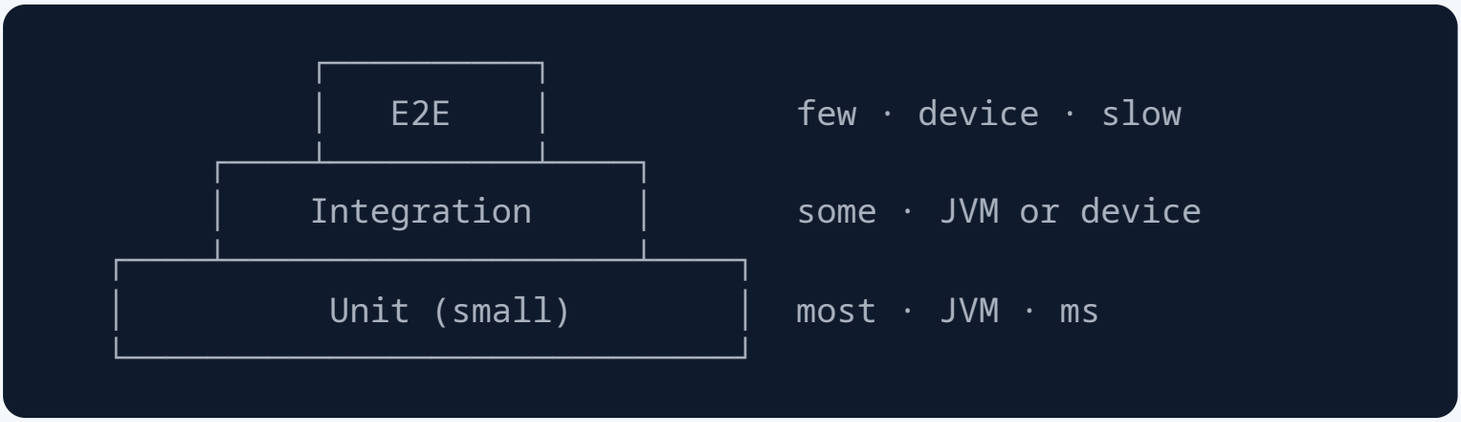
- Compose + UDF made screens a function of state — most logic now runs in **fast JVM tests**
- Coroutines and Flow need **virtual time**, not `Thread.sleep()`
- **Fakes over mocks** is Google's guidance — and the portable choice in KMP `commonTest`, where JVM mocking libraries don't run
- A slow or flaky suite gets skipped; a fast one gets trusted
- Part 2 (Day 10) covers integration, UI, screenshot and performance tests

Level	Questions
Junior	Local vs instrumented · Anatomy of a unit test · Test doubles
Mid	Fakes vs mocks · MockK · Coroutines with runTest · Flows with Turbine
Senior	ViewModel tests in depth · Repository tests · Designing for testability
Architect	Strategy per layer · TDD in practice · Scaling a test suite



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: the test pyramid



- **Scope** (how much code runs) and **environment** (JVM or device) are two separate axes
- Lower = faster, cheaper, more precise failures. Higher = more realistic, slower, flakier
- A test earns its place only if it fails for real bugs — and passes otherwise

THE ONE RULE BEHIND EVERY ANSWER TODAY

Test **behavior**, not implementation. If a refactor that changes no behavior breaks your tests, the tests are coupled to the wrong thing.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 9

Junior

Local vs instrumented · Anatomy of a unit test · Test doubles



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Local vs instrumented tests

	Local — `src/test`	Instrumented — `src/androidTest`
Runs on	JVM on your machine / CI	Device or emulator
Android framework	Stub `android.jar` — or real code via Robolectric	Real
Speed	Milliseconds per test	Seconds, plus install and boot
Typical use	Logic, ViewModels, repositories, screens with Robolectric	Real-device behavior, E2E journeys, system UI
Gradle task	`testDebugUnitTest`	`connectedDebugAndroidTest`

INTERVIEW TRAP

"Unit tests run on the JVM, UI tests run on a device." — That mixes two axes. A Compose screen test can run on the JVM with Robolectric; a small test of Keystore code needs a device. Pick the **scope** first, then the cheapest **environment** with enough fidelity.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Anatomy of a good unit test

```
class DiscountCalculatorTest {
    private val calculator = DiscountCalculator()

    @Test
    fun `orders over 100 get 10 percent off`() {
        // Arrange
        val order = Order(totalCents = 20_000)
        // Act
        val discount = calculator.discountFor(order)
        // Assert
        assertEquals(2_000, discount)
    }
}
```

- **Arrange / Act / Assert** (Given / When / Then): one behavior per test
- The name states the behavior and the expected result — it's the failure message
- Fresh state per test: JUnit creates a **new instance** of the test class for every test method



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What interviewers check

- Test the **public API**. Private helpers are covered through it — no `@VisibleForTesting` just to reach them.
- Deterministic: no real network, clock, randomness or test order.
- Readable over clever: a test is documentation of the behavior.

INTERVIEW TRAP

"One assert per test." — The rule is **one behavior** per test. Several asserts that describe one outcome (all fields of a result) are fine.

INTERVIEW TRAP

Backtick names with spaces work in local JVM tests. In **instrumented** tests they're only supported on devices running API 30+, so many teams use `camelCase_underscores` in `androidTest`.

FOLLOW-UP

"JUnit 4 or 5?" — Instrumented tests run on `AndroidJUnitRunner` (JUnit 4). JUnit 5 works for local tests; on devices only through a third-party plugin.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Test doubles — name them precisely

Double	What it is	Android example
Dummy	Passed in, never used	An `Analytics` parameter the path ignores
Stub	Returns fixed answers, no logic	`every { flags.isOn(NEW_FEED) } returns true`
Fake	Working, lightweight implementation	In-memory repository, fixed `Clock`
Mock	Records calls so the test can verify them	`verify { analytics.log("paid") }`
Spy	Real object, with some calls observed or overridden	`spyk(realFormatter)`
Shadow	Robolectric's stand-in for a framework class	`ShadowLooper`

INTERVIEW TRAP

"In a unit test every dependency must be mocked." — **No.** Replace only what is slow, nondeterministic or outside your control: network, disk, clock, randomness. Value objects, mappers and pure collaborators stay **real**.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 9

Mid-Level

Fakes vs mocks · MockK · Coroutines with runTest · Flows with Turbine



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Fakes: real behavior, zero framework

```
class FakeUserRepository : UserRepository {
    private val users = MutableStateFlow(mapOf<String, User>())
    var failNextRefresh = false

    override fun observe(id: String): Flow<User?> =
        users.map { it[id] }

    override suspend fun refresh(id: String): Result<Unit> {
        if (failNextRefresh) {
            failNextRefresh = false
            return Result.failure(IOException("offline"))
        }
        return Result.success(Unit)
    }

    fun emit(u: User) = users.update { it + (u.id to u) }
}
```

- State persists between calls — tests assert **outcomes**, not call sequences
- Written once, shared by hundreds of tests (a `:core:testing` module)



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Fake or mock?

	Fake	Mock (MockK / Mockito)
Verifies	State and outcomes	Interactions: calls, arguments, order
On refactor	Rarely breaks	Often breaks — tied to implementation
Setup	Written once, reused	`every { }` in every test
KMP `commonTest`	Yes	MockK / Mockito: JVM only

A mock is the right tool when the interaction **is** the behavior: an analytics event sent, a callback fired exactly once.

INTERVIEW TRAP

Mocking types you don't own — Retrofit, OkHttp, Room DAOs, `Context`. The mock encodes your *guess* of their behavior. Wrap them behind your own interface, fake that, and test the real thing in an integration test (Day 10).



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. MockK essentials

```
private val api = mockk<UserApi>()
private val analytics = mockk<Analytics>(relaxUnitFun = true)

@Test fun `refresh stores user and logs event`() = runTest {
    coEvery { api.fetchUser("42") } returns UserDto("42", "Sam")
    val dao = FakeUserDao()
    val repo = UserRepository(api, dao, analytics)

    repo.refresh("42")

    assertEquals("Sam", dao.get("42")?.name)
    coVerify(exactly = 1) { api.fetchUser("42") }
    verify { analytics.log("user_refreshed") }
}
```

- `every` / `coEvery` stub; `verify` / `coVerify` check calls (`co…` for `suspend`)
- `slot<T>()` + `capture(slot)` to inspect arguments; `confirmVerified(mock)` to forbid extra calls



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. MockK pitfalls

INTERVIEW TRAP

`mockk(relaxed = true)` everywhere. Unstubbed calls silently return `0`, `""` or empty lists — the test passes while the code under test does the wrong thing. Relax only `Unit` functions (`relaxUnitFun = true`) when you don't care about them.

INTERVIEW TRAP

`mockkObject` / `mockkStatic` change global state. Forget `unmockkAll()` in `@After` and the mock leaks into the next test. Needing them is usually a sign of a missing seam (Q10).

FOLLOW-UP

"Mockito or MockK?" — MockK is Kotlin-first: `suspend` stubbing, objects, top-level and extension functions. Mockito 5 uses the inline mock maker by default, so final Kotlin classes are mockable too; `mockito-kotlin` adds `whenever`. Both are JVM-only.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Coroutines: `runTest` and virtual time

```
class SyncManager(
    private val api: SyncApi,
    private val io: CoroutineDispatcher = Dispatchers.IO,
) {
    suspend fun syncWithRetry(): Boolean = withContext(io) {
        repeat(3) { attempt ->
            if (api.sync()) return@withContext true
            delay(1_000L * (attempt + 1))
        }
        false
    }
}

@Test fun `gives up after three attempts`() = runTest {
    val api = FakeSyncApi(alwaysFail = true)
    val io = StandardTestDispatcher(testScheduler)
    val sync = SyncManager(api, io)

    assertFalse(sync.syncWithRetry())
    assertEquals(3, api.calls)
    assertEquals(6_000L, currentTime) // virtual ms, instant
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. The rules of `runTest`

- `delay` skips ahead in **virtual time**; `currentTime` shows where the clock is
- New coroutines on a `StandardTestDispatcher` are **queued**: `runCurrent()` runs what's due now, `advanceTimeBy()` moves the clock, `advanceUntilIdle()` drains everything
- **One scheduler for all test dispatchers —** pass `testScheduler`. If `Dispatchers.Main` is a `TestDispatcher`, `runTest` reuses its scheduler
- `backgroundScope` for endless collectors; uncaught exceptions in child coroutines fail the test

INTERVIEW TRAP

"*`runTest` resets `Dispatchers.Main` for you.*" — It doesn't. Call `Dispatchers.setMain()` before and `resetMain()` after each test — or use the `MainDispatcherRule` from Day 3.

INTERVIEW TRAP

A hard-coded `Dispatchers.IO` escapes the test scheduler: `delay` waits in real time and `advanceUntilIdle()` doesn't cover it — slow or flaky tests. **Inject the dispatcher.**



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Flows: Turbine

```
// Under test
fun search(q: String): Flow<SearchResult> =
    repo.results(q)
        .map<List<Item>, SearchResult> { SearchResult.Data(it) }
        .onStart { emit(SearchResult.Loading) }

@Test fun `emits loading then results`() = runTest {
    val repo = FakeSearchRepository() // uses a SharedFlow
    val useCase = SearchUseCase(repo)
    val items = listOf(item1)

    useCase.search("kotlin").test {
        assertEquals(SearchResult.Loading, awaitItem())
        repo.emit(items)
        assertEquals(SearchResult.Data(items), awaitItem())
        cancelAndIgnoreRemainingEvents()
    }
}
```

- `awaitItem()`, `awaitComplete()`, `awaitError()`, `expectNoEvents()`
- Several flows at once: `turbineScope { flow.testIn(backgroundScope) }`



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Flow testing traps

INTERVIEW TRAP

"My StateFlow test should see every state." — `StateFlow` is **conflated** and skips equal values. A collector sees the *latest* value, so a fast `Loading → Success` may show only `Success`. To assert intermediate states, control timing: a fake that suspends until the test releases it, or a `StandardTestDispatcher`.

INTERVIEW TRAP

Hot flows don't replay by default: a `SharedFlow(replay = 0)` event emitted **before** `test { }` subscribes is gone. Subscribe first, then trigger.

- Turbine fails the test if events are left **unconsumed** — consume them, or end with `cancelAndIgnoreRemainingEvents()`
- No Turbine? Collect into a list in `backgroundScope` with `UnconfinedTestDispatcher(testScheduler)`



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 9



Senior

ViewModel tests in depth · Repository tests · Designing for testability



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. ViewModel tests: intermediate states

`ProfileViewModel` loads in `init`, keeps state in a `MutableStateFlow` and has `onRetry()`:

```
@get:Rule
val main = MainDispatcherRule(StandardTestDispatcher())

@Test fun `retry after error shows profile`() = runTest {
    val repo = FakeProfileRepository(failFirst = true)
    val handle = SavedStateHandle(mapOf("userId" to "42"))
    val vm = ProfileViewModel(repo, handle)
    assertEquals(ProfileUiState.Loading, vm.uiState.value)

    advanceUntilIdle()
    assertIs<ProfileUiState.Error>(vm.uiState.value)

    vm.onRetry()
    advanceUntilIdle()
    val ready = vm.uiState.value as ProfileUiState.Ready
    assertEquals("42", ready.user.id)
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Choosing the test dispatcher

	`UnconfinedTestDispatcher`	`StandardTestDispatcher`
New coroutines	Start eagerly	Queued until you advance
Good for	Collectors, "final state" tests	Asserting `Loading`, ordering, timing
Risk	Hides ordering bugs	Forget to advance → nothing runs

- `SavedStateHandle(mapOf(…))` simulates **restored** state after process death — no device needed
- `stateIn(WhileSubscribed)` state needs a collector (Day 3) — or `uiState.test { }`

INTERVIEW TRAP

With an unconfined Main and a fake that returns instantly, `init` finishes inside the constructor: the first value you read is already `Error`, never `Loading`. That's not a bug in the ViewModel — it's the dispatcher you chose.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Repository tests: the offline-first contract

```
@Test fun `failed refresh keeps cached articles`() = runTest {
    val dao = FakeArticleDao(initial = listOf(cachedEntity))
    val api = FakeArticleApi(error = IOException("offline"))
    val repo = ArticleRepository(dao, api)

    val result = repo.refresh()

    assertTrue(result.isFailure)
    assertEquals(
        listOf(cachedEntity.toDomain()),
        repo.observeArticles().first(),
    )
}
```

Cover the scenarios, not the lines: **cache + success**, **cache + failure**, **empty + failure**, **mapping** of DTOs and errors.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What makes repository tests valuable

- Fake the **data sources** (DAO, API), keep the repository's own logic real
- Assert what the caller sees: emitted data and the returned `Result` — not which DAO method was called
- Test the error **mapping**: `IOException` → a domain error the UI can show

INTERVIEW TRAP

`verify { dao.upsertAll(any()) }` as the only assertion. It proves a call happened, not that the data is right — and it breaks when you switch to a transaction or a different DAO method.

INTERVIEW TRAP

Treating cancellation as a failure. If `refresh()` catches `CancellationException` (e.g. with `runCatching`), a cancelled refresh returns `Result.failure` and the caller shows an error for a screen the user already left. Test it: make the fake API suspend, cancel the caller, assert no error was recorded.

FOLLOW-UP

"The fake DAO passes — is the SQL correct?" — The fake proves repository logic only. Queries, conflict strategies and migrations need a test against real Room (in-memory) — Day 10.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Designing for testability: add seams

Hard to test	Seam
Hard-coded `Dispatchers.IO`	Inject a `CoroutineDispatcher`
`System.currentTimeMillis()`, `Instant.now()`	Inject `java.time.Clock`; `Clock.fixed()` in tests
`UUID.randomUUID()`, `Random`	Inject an ID generator / seeded `Random`
`object` singletons with state	Interface + DI
`Context` / resources in a ViewModel	Resource IDs in UI state, or a small interface
`android.util.Log` in JVM tests	A logger interface

A seam is a place where a test can swap behavior **without editing the code under test**. Constructor injection is the simplest seam.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. A clock is a dependency

```
class TokenStore(
    private val prefs: TokenPrefs,
    private val clock: Clock,
) {
    suspend fun isExpired(): Boolean =
        prefs.expiresAt() <= clock.instant()
}

@Test fun `token is expired exactly at expiry`() = runTest {
    val expiry = Instant.parse("2026-01-01T10:00:00Z")
    val store = TokenStore(
        prefs = FakeTokenPrefs(expiresAt = expiry),
        clock = Clock.fixed(expiry, ZoneOffset.UTC),
    )
    assertTrue(store.isExpired())
}
```

INTERVIEW TRAP

"Just use `mockkStatic` *for the time call."* — It works, but it patches global state and hides the design problem. And `unitTests.isReturnDefaultValues = true` silences "Method ... not mocked" errors from `android.jar` — along with real bugs.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 9

→

Architect

Strategy per layer · TDD in practice · Scaling a test suite



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. A test strategy per layer

What	Test	Doubles	Runs on
Business rules, mappers, reducers	Unit	None	JVM
ViewModels, use cases	Unit	Fake repos, test dispatchers	JVM
Repositories	Unit	Fake data sources	JVM
DAO SQL, JSON, API contract	Integration	In-memory Room, MockWebServer	JVM or device
Screens	UI + screenshot	Fake state	JVM (Robolectric) or device
Critical journeys: login, pay	E2E	Fake or staging backend	Device

Spend the budget where bugs are **expensive** and logic is **dense** — a payment flow deserves more than a settings screen.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Pyramid traps

INTERVIEW TRAP

"Unit tests need 70–90% coverage, integration 10–20%." — That mixes up two ideas. The often-quoted **70/20/10** is a guideline for how many tests of each **type** to write. Coverage is how much code any test executes — a different metric.

INTERVIEW TRAP

"100% coverage means the app is tested." — Coverage shows what **ran**, not what was **checked**. A test with no assertion still adds coverage.

FOLLOW-UP

"Pyramid or testing trophy?" — The shape matters less than the principle: most confidence per second of test time. With Robolectric and screenshot tests, many "UI" tests now run on the JVM, so the middle of the pyramid grew cheaper.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. TDD in practice

```
// 1. RED — reproduce the bug report as a test
@Test fun `iban with spaces is valid`() {
    val iban = "DE89 3704 0044 0532 0130 00"
    assertTrue(IbanValidator.isValid(iban))
}

// 2. GREEN — the smallest change that passes
fun isValid(raw: String): Boolean {
    val iban = raw.replace(" ", "")
    return iban.length in 15..34 && checksumOk(iban)
}

// 3. REFACTOR — clean up; the test is your safety net
```

TDD fits well	TDD fits poorly
Pure logic, parsers, reducers, validators	Exploratory spikes, unknown APIs
Every bug fix — test first, then fix	Visual layout (use screenshot tests)

INTERVIEW TRAP

"TDD means writing all tests up front." — It's a tight loop: **one** failing test, make it pass, refactor, repeat.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Scaling the suite: fakes you can trust

```
abstract class UserRepositoryContract {
    abstract fun createRepository(): UserRepository

    @Test fun `saved user can be read back`() = runTest {
        val repo = createRepository()
        repo.save(user42)
        assertEquals(user42, repo.observe("42").first())
    }
}

// src/test — the fake must honour the same contract
class FakeUserRepositoryTest : UserRepositoryContract() {
    override fun createRepository() = FakeUserRepository()
}
// Robolectric or androidTest — same tests, real Room impl
```

- Shared fakes live in one `:core:testing` module — one fake per interface, not one per test
- **Contract tests** run the same suite against fake and real implementation, so fakes can't drift



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Measuring test health

Signal	Tells you	Watch out
Line / branch coverage (Kover, JaCoCo)	Code no test ever runs	High coverage ≠ good assertions
Mutation score (e.g. PIT)	Whether tests catch injected bugs	Slow — run on core modules
Suite duration	Feedback speed	Slow suites get skipped
Flaky rate	Trust in red builds	Day 10
Escaped defects	What tests missed	Each one becomes a new test

SENIOR ANSWER IN ONE LINE

"Test behavior at the lowest level that can catch the bug, with fakes you can trust, and keep the suite fast enough that nobody skips it."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 9 · ARCHITECT

Day 9 — cheat sheet

- Scope (unit / integration / E2E) and environment (JVM / device) are **separate axes**.
- One behavior per test, Arrange / Act / Assert, names that read as failure messages.
- Replace only slow or nondeterministic dependencies — **fakes over mocks**.
- Mocks when the interaction *is* the behavior; never mock types you don't own.
- `runTest` = virtual time; one scheduler for all test dispatchers; it does **not** reset Main.
- Inject dispatchers, clocks and ID generators — seams beat `mockkStatic`.
- `StateFlow` conflates: control timing to assert `Loading`; Turbine for flow sequences.
- Repository tests check the offline-first contract and never turn cancellation into an error.
- 70/20/10 is about test **types**, not coverage; coverage ≠ quality.
- Contract tests keep shared fakes honest.

Day 10 is next → Testing Strategies, Part 2: integration, UI & screenshot tests. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700